

```
In [ ]: import marimo as mo
import plotly.graph_objects as go
import polars as pl
from pathlib import Path
```

1. Data loading

```
In [ ]: PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "mlcasestudy Final.csv"
```

```
In [ ]: df = pl.read_csv(DATA_PATH, try_parse_dates=True).with_columns(
    (pl.col("amount_outstanding_14d") > 0).alias("default_14d"),
    (pl.col("amount_outstanding_21d") > 0).alias("default_21d"),
)

row_count = df.height
column_count = df.width
default_rate = df.select(pl.mean("default_21d")).item()
```

```
In [ ]: mo.md(f"""
The dataset consists of {row_count:,} rows and {column_count:,} columns, with
""")
```

The dataset consists of 111,902 rows and 26 columns, with a 21-day default rate of 5.4%.

```
In [ ]: df
```

Q Search...

loan_id str unique: 111,902	loan_issue_date date min: Jun 1, 2023 max: Oct 27, 2023 unique:	loan_amount i64 min: 102 max: 392,913 unique: 35,667	amount_outstanding_14d i64 min: 0 max: 193,313 unique: 7,687	amount_outstanding_21d i64 min: 0 max: 193,313 unique: 4,526	card_expiry_month i64 (nulls: 155) min: 1 max: 12 unique: 13	card_expiry_year i64 (nulls: 155) min: 2,023 max: 2,032 unique: 11
a55cf3062bd45a83766096fd0c4697d6	2023-06-24	5,300	0	0	2	
45e3d15cb313d767470722e952e14313	2023-09-22	9,547	7,161	7,161	6	
b195c127c7605b7b936b90bcd5e889c	2023-07-03	5,948	0	0	12	
36028b1dbeb4aef565a79ca9700667a	2023-08-19	15,871	11,904	0	2	
5317f8e22a8790b6e11a0f12320422d	2023-09-30	19,510	0	0	8	
db9fce76456e79737ad5fb120b77fcd	2023-10-03	38,305	0	0	9	
c7c3a8bf87bba0fae8009f795ee4a18c	2023-10-27	6,120	0	0	6	
bcaa16391c5c12b9e2d145f543742ccb	2023-10-03	3,658	0	0	10	
bea984a75f214b5cd8a6b8b3f2a241a0	2023-10-09	24,587	0	0	5	
e466816902e6048fb3a3e96ff35bb01f	2023-07-07	22,050	0	0	3	

111,902 rows, 26 columns No selection

Each row represents a loan, identified by `loan_id` and dated by `loan_issue_date`; we know the initial amount of the loan and the outstanding after 14 and 21 days, which are the two deadlines for repayment.

Next we have information on the current card associated with the loan, and on the credit history of the consumer.

Finally, the last two columns describe the merchant the financed purchase is associated to.

```
In [ ]: categorical_columns = ["merchant_group", "merchant_category"]
numeric_columns = [
    column for column, dtype in df.schema.items() if dtype.is_numeric()
]
```

2. Data understanding

2.1 Date coverage

```
In [ ]: date_range = df.select(
    pl.min("loan_issue_date").alias("min_loan_issue_date"),
    pl.max("loan_issue_date").alias("max_loan_issue_date"),
    pl.col("loan_issue_date").n_unique().alias("distinct_issue_dates"),
)
```

```
In [ ]: date_range
```

Search...

min_loan_issue_date date	max_loan_issue_date date	distinct_issue_dates u32	
2023-06-01	2023-10-27	149	

1 row, 3 columns

No selection

2.2 Target definition and prevalence

For the moment, let's focus on the 21-days target which is the hardest deadline for repayment.

```
In [ ]: target_df = df.with_columns(
    (pl.col("amount_outstanding_14d") > 0).alias("default_14d"),
    (pl.col("amount_outstanding_21d") > 0).alias("default_21d"),
)

target_df.select(
    pl.mean("default_14d").alias("default_rate_14d"),
    pl.mean("default_21d").alias("default_rate_21d"),
)
```

Search...

default_rate_14d f64	default_rate_21d f64	
0.1287197726582188	0.054413683401547785	

1 row, 2 columns

No selection

```
In [ ]: target_summary = (
    target_df.group_by("default_21d")
    .agg(
        pl.len().alias("loans"),
        pl.sum("loan_amount").alias("loan_amount_total"),
        pl.mean("loan_amount").round(2).alias("avg_loan_amount"),
        pl.mean("amount_outstanding_21d").round(2).alias("avg_outstanding_21d"),
    )
)
```

```

.with_columns(
    pl.when(pl.col("default_21d"))
    .then(pl.lit("Default at 21 days"))
    .otherwise(pl.lit("Repaid by 21 days"))
    .alias("target_label"),
    (pl.col("loans") / pl.sum("loans")).round(4).alias("loan_share"),
)
.select(
    "target_label",
    "default_21d",
    "loans",
    "loan_share",
    "loan_amount_total",
    "avg_loan_amount",
    "avg_outstanding_21d",
)
.sort("default_21d")
)

```

In []: target_summary

Search...

target_label str	default_21d bool	loans u32	loan_share f64	loan_amount_total i64	avg_loan_amount f64	avg_outstanding_21d f64
Repaid by 21 days	False	105,813	0.9456	1,853,918,755	17,520.71	0.00
Default at 21 days	True	6,089	0.0544	112,699,914	18,508.77	7,854.91

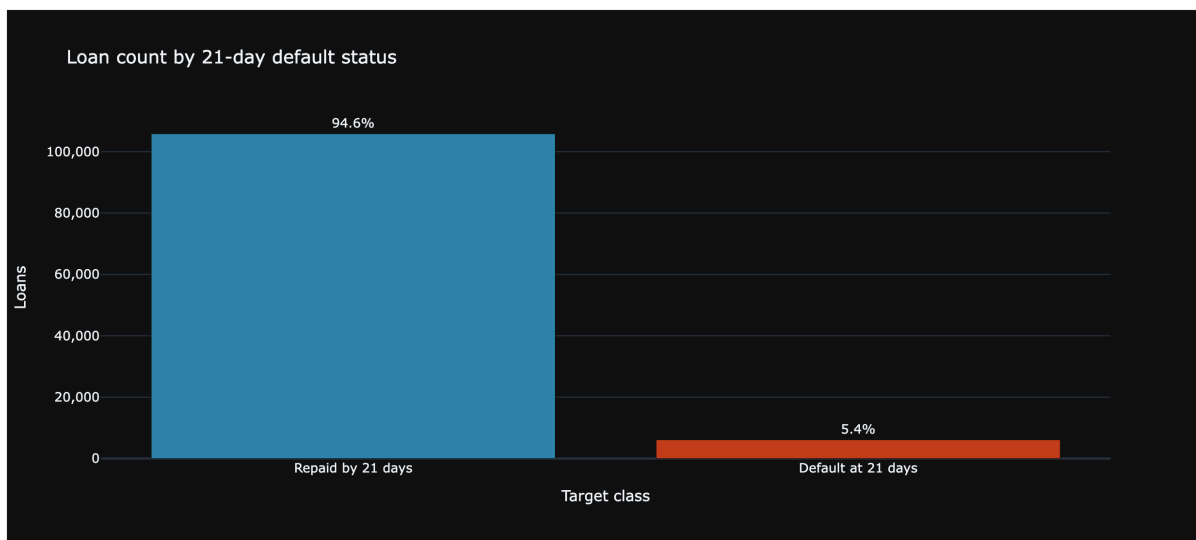
2 rows, 7 columns No selection

Defaulted loans amount to about 5% and do not seem to differ significantly by their amount; on average, the outstanding at default time is a bit less than 50% the initial amount.

```

In [ ]: _fig = go.Figure(
    go.Bar(
        x=target_summary["target_label"].to_list(),
        y=target_summary["loans"].to_list(),
        text=[f"{value:.1%}" for value in target_summary["loan_share"].to_list()],
        textposition="outside",
        marker_color=["#2E86AB", "#C73E1D"],
    )
)
_fig.update_layout(
    title="Loan count by 21-day default status",
    xaxis_title="Target class",
    yaxis_title="Loans",
    yaxis_tickformat="," ,
    showlegend=False,
    height=500,
)
mo.ui.plotly(_fig)

```



2.3 Numeric feature profile

```
In [ ]: numeric_profile = pl.concat(
    [
        df.select(
            pl.lit(column).alias("column"),
            pl.col(column).count().alias("non_null_count"),
            pl.col(column).null_count().alias("missing_count"),
            pl.col(column).min().alias("min"),
            pl.col(column).quantile(0.25, interpolation="nearest").alias("p25"),
            pl.col(column).median().alias("median"),
            pl.col(column).mean().round(2).alias("mean"),
            pl.col(column).quantile(0.75, interpolation="nearest").alias("p75"),
            pl.col(column).quantile(0.95, interpolation="nearest").alias("p95"),
            pl.col(column).max().alias("max"),
            pl.col(column).std().round(2).alias("std"),
        )
        for column in numeric_columns
    ],
    how="vertical",
)
```

```
In [ ]: numeric_profile
```

Search...

column str unique: 20	non_null_count u32	missing_count u32	min i64	p25 f64	median f64	mean f64	p75 f64	p95 f64	max i64
loan_amount	111,902	0	102	6,160	10,877	17,574.47	19,461	55,075	392
amount_outstanding_14d	111,902	0	0	0	0	783.98	0	4,360	193
amount_outstanding_21d	111,902	0	0	0	0	427.41	0	937	193
card_expiry_month	111,747	155	1	4	6	6.56	9	12	
card_expiry_year	111,747	155	2,023	2,025	2,026	2,026.08	2,027	2,028	2
existing_klarna_debt	82,690	29,212	-220,618	0	1,404	16,018.21	19,008	71,976	812
num_active_loans	111,902	0	0	0	0	0.70	1	3	
days_since_first_loan	111,902	0	-1	-1	136	294.85	546	1,008	1
new_exposure_7d	111,902	0	0	0	0	3,555.79	0	20,525	562
new_exposure_14d	111,902	0	0	0	0	4,895.85	2,142	26,484	562

20 rows, 11 columns

No selection

```
In [ ]: numeric_feature_dropdown = mo.ui.dropdown(
        options=numeric_columns,
        value=numeric_columns[0],
        label="Select numeric feature",
    )
```

```
In [ ]: _selected_numeric_feature = numeric_feature_dropdown.value
        _distribution_plot_df = df.select(
            pl.col(_selected_numeric_feature).alias("value")
        ).drop_nulls()

        _distribution_values = _distribution_plot_df["value"].to_list()

        _distribution_fig = go.Figure()

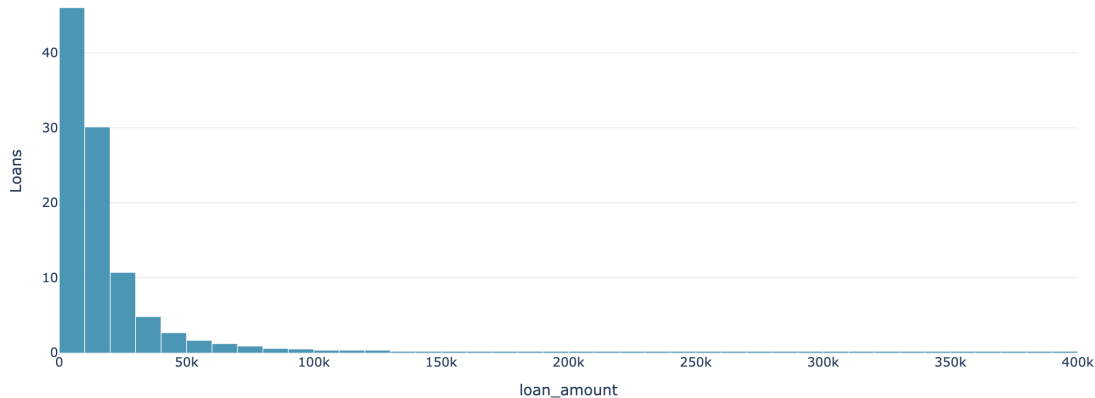
        _distribution_fig.add_trace(
            go.Histogram(
                x=_distribution_values,
                name=_selected_numeric_feature,
                nbinsx=60,
                marker_color="#2E86AB",
                opacity=0.85,
                histnorm="percent",
                hovertemplate=(
                    f"{_selected_numeric_feature}: %{{x}}<br>Count: %{{y:,.}}<extra><
                ),
            )
        )

        _distribution_fig.update_layout(
            title=f"Distribution of {_selected_numeric_feature}",
            axis_title=_selected_numeric_feature,
            axis_title="Loans",
            axis_tickformat="",
            axis2={
                "visible": False,
                "overlaying": "y",
                "range": [0, 1],
            },
            bargap=0.03,
            height=520,
            showlegend=False,
            template="plotly_white",
        )

        mo.vstack(
            [
                numeric_feature_dropdown,
                mo.ui.plotly(_distribution_fig),
            ]
        )
```

Select numeric feature `loan_amount` ▼

Distribution of loan_amount



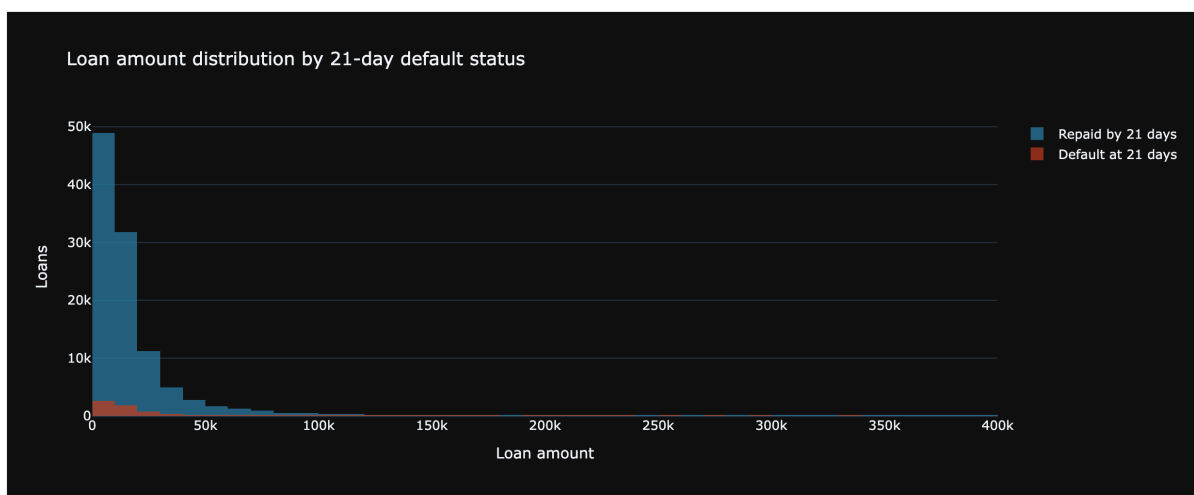
Some considerations on the distributions of the numeric features:

- `loan_amount` shows a skewed distribution akin to an inverse power law, with a thin long tail of high-value loans; `amount_outstanding_14d` and `amount_outstanding_21d` are even more skewed, but they contain a more complex mixture of zeroes (for fully repaid loans) and positive values with a similar long-tail pattern, which we will study later in more detail.
- `card_expiry_month` and `card_expiry_year` display what we would expect: uniform distribution for month, and a bell-like curve centered around 2-4 years after the loan issue dates in the dataset (all from 2023) and with a small tail.
- `existing_klarna_debt` is also right-skewed like the "amount" columns, but it also has some negative values. Since we don't have information on their meaning, we will probably clamp them to zero.
- `num_active_loans` shows that most loans in the dataset occur as first loans (zero active loans at the time of issue), but there are also a few customers with high counts that might be worth investigating, since having so many active loans could be a sign of financial distress or reckless behavior.
- `days_since_first_loan` has a surprising 32% of "-1" values, all associated with "0" or "null" `existing_klarna_debt`. To be investigated further.
- `new_exposure_7d`, `new_exposure_14d`, `num_confirmed_payments_3m` and `num_confirmed_payments_6m` show once again the typical right-skewed shape of other numeric columns.
- `num_failed_payments_3m`, `num_failed_payments_6m` and `num_failed_payments_1y` all have more than 95% of zeroes, which suggest that failed payments are relatively rare events in the customer history, but they might still be important predictors of default risk.
- finally, all the `amount_repaid` columns are once again right-skewed with a large share of zeroes, which is consistent with the fact that many customers do not have

any repayment history with Klarna, but for those who do, the amounts can vary widely and reach high values.

```
In [ ]: _repaid = df.filter(~df["default_21d"])["loan_amount"].to_list()
        _defaulted = df.filter(df["default_21d"])["loan_amount"].to_list()

        _fig = go.Figure()
        _fig.add_trace(
            go.Histogram(
                x=_repaid,
                name="Repaid by 21 days",
                opacity=0.7,
                marker_color="#2E86AB",
                nbinsx=60,
            )
        )
        _fig.add_trace(
            go.Histogram(
                x=_defaulted,
                name="Default at 21 days",
                opacity=0.7,
                marker_color="#C73E1D",
                nbinsx=60,
            )
        )
        _fig.update_layout(
            title="Loan amount distribution by 21-day default status",
            xaxis_title="Loan amount",
            yaxis_title="Loans",
            barmode="overlay",
            height=460,
        )
        mo.ui.plotly(_fig)
```



2.4 Outstanding balances

```
In [ ]: outstanding_profile = df.select(
        pl.col("amount_outstanding_14d").mean().round(2).alias("avg_outstanding_14d")
    )
```

```

pl.col("amount_outstanding_21d").mean().round(2).alias("avg_outstanding_21d"),
(pl.col("amount_outstanding_14d") > 0)
.mean()
.round(4)
.alias("share_outstanding_14d"),
(pl.col("amount_outstanding_21d") > 0)
.mean()
.round(4)
.alias("share_outstanding_21d"),
(pl.col("amount_outstanding_14d") - pl.col("amount_outstanding_21d"))
.mean()
.round(2)
.alias("avg_balance_reduction_14d_to_21d"),
)

```

In []: outstanding_profile

Q Search...

avg_outstanding_14d f64	avg_outstanding_21d f64	share_outstanding_14d f64	share_outstanding_21d f64	avg_balance_reduction_14d_to_21d f64
783.98	427.41	0.1287	0.0544	356.57

1 row, 5 columns No selection

We defined the target at 21 days, preferring it over the one at 14 days because it represented the last deadline before default.

Nonetheless, it may be also useful to predict 14-days default, as an extra signal that may prompt some action towards the customer.

The higher default rate at 14 days may amount to an easier prediction task, too.

From the data aggregated above, though, we can see that on average almost half of the outstanding gets recovered in the extra week granted by klarna before declaring a default; this suggests that the extra week is indeed useful and that leaving the threshold at 21 days may be the right choice.

Of course an appropriate assessment of this threshold should include the economic effects that this extra time has on klarna.

2.5 Timing effect

```

In [ ]: monthly_profile = (
    df.group_by(pl.col("loan_issue_date").dt.truncate("1mo").alias("month"))
    .agg(
        pl.len().alias("loans"),
        pl.mean("default_21d").round(4).alias("default_rate"),
        pl.mean("loan_amount").round(2).alias("avg_loan_amount"),
        pl.sum("loan_amount").alias("loan_amount_total"),
    )
    .sort("month")
)

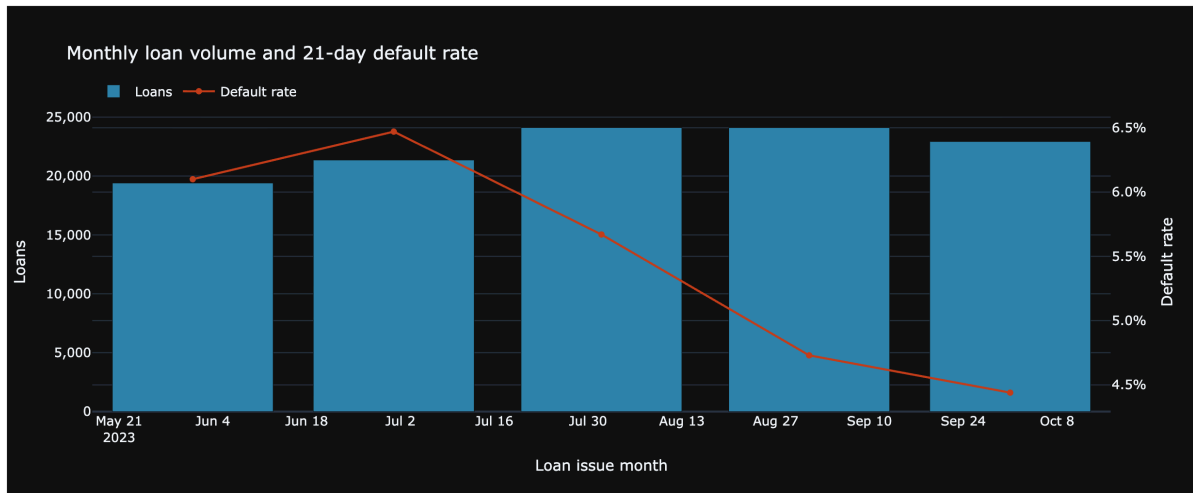
```



```

In [ ]: _fig = go.Figure()
_fig.add_trace(
    go.Bar(
        x=monthly_profile["month"].to_list(),
        y=monthly_profile["loans"].to_list(),
        name="Loans",
        marker_color="#2E86AB",
        yaxis="y",
    )
)
_fig.add_trace(
    go.Scatter(
        x=monthly_profile["month"].to_list(),
        y=monthly_profile["default_rate"].to_list(),
        name="Default rate",
        mode="lines+markers",
        marker_color="#C73E1D",
        yaxis="y2",
    )
)
_fig.update_layout(
    title="Monthly loan volume and 21-day default rate",
    xaxis_title="Loan issue month",
    yaxis={"title": "Loans", "tickformat": ","},
    yaxis2={
        "title": "Default rate",
        "tickformat": ".1%",
        "overlying": "y",
        "side": "right",
    },
    legend={
        "orientation": "h",
        "yanchor": "bottom",
        "y": 1.02,
        "xanchor": "left",
        "x": 0,
    },
    height=460,
)
mo.ui.plotly(_fig)

```



There seems to be a significant trend in the default rate, which decreases as time passes while the sample size is mostly constant if not increasing.

Is this actually a trend? a seasonal effect? random fluctuations? The time span of provided data is too short to tell, but it would definitely merit further investigations.

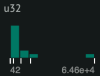
2.6 Merchant dimensions

```
In [ ]: merchant_group_profile = (
    df.groupby("merchant_group")
    .agg(
        pl.len().alias("loans"),
        pl.mean("default_21d").round(4).alias("default_rate"),
        pl.mean("loan_amount").round(2).alias("avg_loan_amount"),
        pl.sum("loan_amount").alias("loan_amount_total"),
    )
    .sort("loans", descending=True)
)

merchant_category_profile = (
    df.groupby("merchant_category")
    .agg(
        pl.len().alias("loans"),
        pl.mean("default_21d").round(4).alias("default_rate"),
        pl.mean("loan_amount").round(2).alias("avg_loan_amount"),
    )
    .sort("loans", descending=True)
)
```

```
In [ ]: merchant_group_profile
```

Q Search...

merchant_group str unique: 13	loans u32	default_rate f64	avg_loan_amount f64	loan_amount_total i64
				
Clothing & Shoes	64,623	0.0587	12,470.58	805,886,404
Intangible products	15,728	0.0626	40,525.79	637,389,678
Leisure, Sport & Hobby	9,397	0.0343	12,864.19	120,884,818
Jewelry & Accessories	8,067	0.0371	15,201.34	122,629,246
Home & Garden	4,090	0.0396	25,982.52	106,268,513
Health & Beauty	3,680	0.0527	13,668.54	50,300,209
Electronics	2,193	0.0438	29,716.07	65,167,343
Food & Beverage	2,088	0.0761	12,934.57	27,007,384
Unknown	1,284	0.0350	13,153.06	16,888,529
Children Products	327	0.0428	13,748.03	4,495,607

13 rows, 5 columns

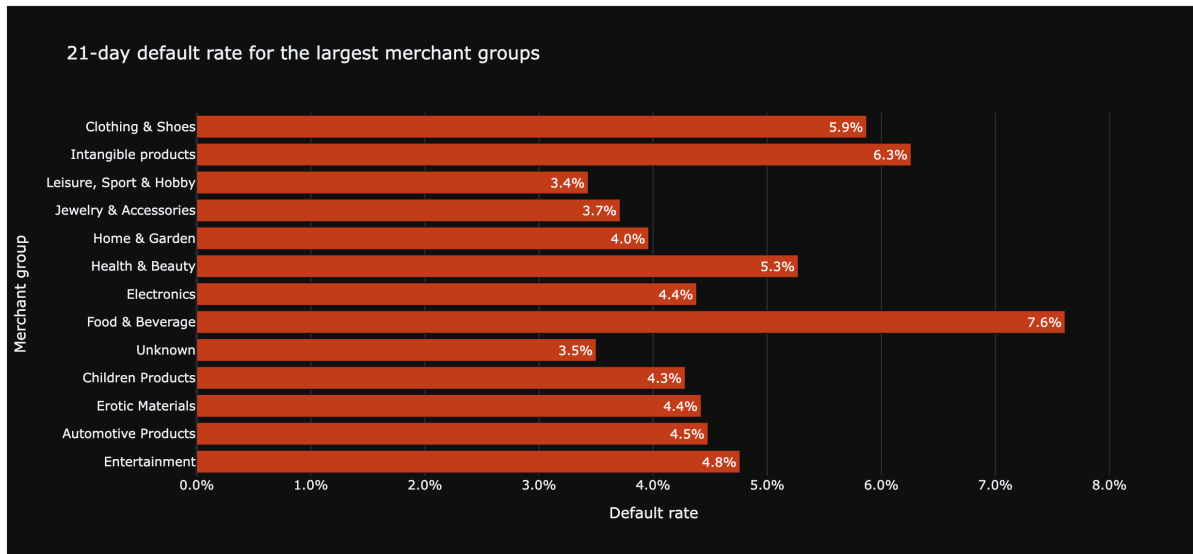
No selection

Merchant groups seem to be quite heterogeneous in number of loans, default rate and avg loan amounts.

Each merchant group showcasing different patterns in consumer habits would be typical; as a predictor, it's definitely a promising feature that can be encoded as categorical or possibly with some clever numerical embedding.

If this was not enough, a possible strategy would be dealing with at least some of the main groups separately, to make sure they get tailored predictions.

```
In [ ]: _plot_df = merchant_group_profile.head(15).sort("loans")
_fig = go.Figure(
    go.Bar(
        x=_plot_df["default_rate"].to_list(),
        y=_plot_df["merchant_group"].to_list(),
        orientation="h",
        marker_color="#C73E1D",
        text=[f"{value:.1%}" for value in _plot_df["default_rate"].to_list()],
        customdata=_plot_df["loans"].to_list(),
        hovertemplate="%{y}<br>Default rate: %{x:.2%}<br>Loans: %{customdata}"
    )
)
_fig.update_layout(
    title="21-day default rate for the largest merchant groups",
    xaxis_title="Default rate",
    xaxis_tickformat=".1%",
    yaxis_title="Merchant group",
    height=520,
)
mo.ui.plotly(_fig)
```



```
In [ ]: categorical_profile = pl.DataFrame(
    {
        "column": categorical_columns,
        "distinct_values": [
            df.select(pl.col(column).n_unique()).item()
            for column in categorical_columns
        ],
        "missing_count": [
            df.select(pl.col(column).null_count()).item()
            for column in categorical_columns
        ],
        "top_value": [
            df.group_by(column)
                .len()
                .sort("len", descending=True)
                .select(column)
                .item(0, 0)
            for column in categorical_columns
        ],
        "top_value_count": [
            df.group_by(column)
                .len()
                .sort("len", descending=True)
                .select("len")
                .item(0, 0)
            for column in categorical_columns
        ],
    }
).with_columns(
    (pl.col("top_value_count") / df.height).round(4).alias("top_value_share")
)
```

```
In [ ]: categorical_profile
```

Q Search...

column str	distinct_values i64	missing_count i64	top_value str	top_value_count i64	top_value_share f64
merchant_group	13	0	Clothing & Shoes	64,623	0.5775
merchant_category	63	0	General Shoes & Clothing	37,875	0.3385

2 rows, 6 columns No selection

```
In [ ]: merchant_category_profile.head(25)
```

Q Search...

merchant_category str unique: 25	loans u32	default_rate f64	avg_loan_amount f64
			
General Shoes & Clothing	37,875	0.0530	12,178.81
Youthful Shoes & Clothing	17,621	0.0805	11,379.20
Adult Shoes & Clothing	9,072	0.0405	15,812.94
Event - Broker & Agencies	6,499	0.0465	27,436.89
Travel - Accommodation & Resorts	6,314	0.0744	60,729.55
Diversified Jewelry & Accessories	6,143	0.0329	14,217.06
Concept Stores & Miscellaneous	4,536	0.0335	9,899.41
Sports Gear & Outdoor	4,283	0.0360	15,621.02
Food & Beverage	2,055	0.0769	12,951.08
Furniture	1,699	0.0359	42,894.18

25 rows, 4 columns No selection

The sub-division, "merchant category", starts having way too many unique values to be dealt properly as a categorical; moreover, there's way too many categories that are very small and would have to be clumped together.

3. Data quality

This section checks whether the dataset is internally consistent enough to support modelling.

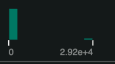
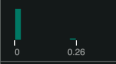
The checks focus on missingness, uniqueness, valid ranges, monotonic time windows, and target leakage considerations.

```
In [ ]: missingness = (
    pl.DataFrame(
        {
            "column": df.columns,
            "missing_count": [
                df.select(pl.col(column).null_count()).item()
                for column in df.columns
            ],
        }
    )
    .with_columns(
        (pl.col("missing_count") / df.height).round(4).alias("missing_share")
    )
    .sort(["missing_count", "column"], descending=[True, False])
)
```

3.1 Missing values

```
In [ ]: missingness
```

Q Search...

column	missing_count	missing_share	
str	i64	f64	
unique: 26			
existing_klarna_debt	29,212	0.2610	
card_expiry_month	155	0.0014	
card_expiry_year	155	0.0014	
amount_outstanding_14d	0	0.0000	
amount_outstanding_21d	0	0.0000	
amount_repaid_14d	0	0.0000	
amount_repaid_1m	0	0.0000	
amount_repaid_1y	0	0.0000	
amount_repaid_3m	0	0.0000	
amount_repaid_6m	0	0.0000	

26 rows, 3 columns No selection

As we already noticed earlier, `existing_klarna_debt` is the only column having a substantial amount of missing values.

About `card_expiry_month` and `card_expiry_year`, the fact that the share of missing values is so small and that these columns appeared to be among the least interesting ones, makes us conclude that we can confidently ignore them for now.

3.2 Uniqueness

```
In [ ]: duplicate_row_count = int(df.is_duplicated().sum())
duplicate_loan_ids = (
    df.groupby("loan_id")
      .len()
      .filter(pl.col("len") > 1)
      .sort("len", descending=True)
)
duplicate_loan_id_rows = duplicate_loan_ids.select(pl.sum("len")).item() or

uniqueness_summary = pl.DataFrame(
    {
        "check": [
            "duplicate full rows",
            "loan_id values appearing more than once",
            "rows with duplicated loan_id",
        ],
        "count": [
            duplicate_row_count,
            duplicate_loan_ids.height,
            duplicate_loan_id_rows,
        ],
    },
)
```

```
In [ ]: uniqueness_summary
```

Q Search...

check str	count i64	
duplicate full rows	0	
loan_id values appearing more than once	0	
rows with duplicated loan_id	0	

3 rows, 2 columns

No selection

No duplicate rows, that's good.

3.3 Range and consistency checks

Based off our understanding of data, we can enforce some domain-driven checks on ranges, signs, consistency among columns, etc.

```
In [ ]: def count_where(predicate: pl.Expr) -> int:
        return int(df.select(predicate.fill_null(False).sum()).item())
```

```
In [ ]: # negative loan amount
count_where(pl.col("loan_amount") <= 0)
```

0

```
In [ ]: # outstanding at 14 days negative or higher than initial amount
count_where(
    (pl.col("amount_outstanding_14d") < 0)
    | (pl.col("amount_outstanding_14d") > pl.col("loan_amount"))
)
```

0

```
In [ ]: # outstanding at 21 days negative or higher than initial amount
count_where(
    (pl.col("amount_outstanding_21d") < 0)
    | (pl.col("amount_outstanding_21d") > pl.col("loan_amount"))
)
```

0

```
In [ ]: # outstanding at 21 days higher than outstanding at 14 days
count_where(pl.col("amount_outstanding_21d") > pl.col("amount_outstanding_14d"))
```

0

```
In [ ]: # card expiry date not a valid future date
count_where(
    ~pl.col("card_expiry_month").is_between(1, 12, closed="both")
    | ~pl.col("card_expiry_month")
    > 2022
)
```

0

```
In [ ]: # negative existing debt
count_where(pl.col("existing_klarna_debt") < 0)
```

59

As we already noticed when looking at distributions, negative values for debt don't have a clear cut explanation and should be investigated further. since they are so few, we can safely clamp them to zero for our purposes.

```
In [ ]: # negative number of loans
count_where(pl.col("num_active_loans") < 0)
```

0

```
In [ ]: # negative number of days since first loan
count_where(pl.col("days_since_first_loan") < 0)
```

36115

```
In [ ]: df.filter(pl.col("days_since_first_loan") < 0)
```

Search...

loan_id str unique: 36,115	loan_issue_date date min: Jun 1, 2023 max: Oct 27, 2023 unique:	loan_amount i64 min: 110 max: 392,333 unique: 21,604	amount_outstanding_14d i64 min: 0 max: 175,657 unique: 3,809	amount_outstanding_21d i64 min: 0 max: 175,657 unique: 2,093	card_expiry_month i64 (nulls: 10) min: 1 max: 12 unique: 13	card_expiry_year i64 (nulls: 10) min: 2,023 max: 2,031 unique: 10
45e3d15cb313d767470722e952e14313	2023-09-22	9,547	7,161	7,161	6	
b195c127c7605b7b936b90bcd5e889c	2023-07-03	5,948	0	0	12	
57a4b8561d30a9a6c18f65fa87d27800	2023-10-20	44,384	0	0	8	
16919070826874d9b90174e2fa170a2d	2023-10-23	5,022	0	0	1	
cb7684f9f55a8da592e4cae775d03dd0	2023-07-21	4,922	2,462	0	6	
73c97efb94b597156d36ba97555812b3	2023-06-18	8,374	0	0	11	
09406b0e3b2f992b7f849dc82647f923	2023-06-02	2,625	0	0	12	
eb78b2106802c354e4e58869a83946fd	2023-09-27	7,950	0	0	8	
3ee8e1aee4358399549af45d9e57f49	2023-09-30	8,105	0	0	12	
07dded3bd113bc276c81b9471a1e3efc	2023-07-23	13,125	3,282	0	7	

36,115 rows, 26 columns

No selection

Looks like `days_since_first_loan` is set to -1 whenever `num_active_loans` is zero. Is this true?

```
In [ ]: df.filter(
    (pl.col("days_since_first_loan") == 0) & (pl.col("num_active_loans") ==
)
```


Search...

loan_id str unique: 202	loan_issue_date date	loan_amount i64	amount_outstanding_14d i64	amount_outstanding_21d i64	card_expiry_month i64 (nulls: 1)	card_expiry_year i64 (nulls: 1)
d04a25362adb7afd2b31b6eb9c85b012	2023-07-23	21,001	0	0	8	
91b13c74b6ee10d6688e0d3b38dd537b	2023-06-23	38,506	0	0	1	
18eea2826408ca03f4c237f466dcccab4	2023-08-19	60,956	0	0	1	
aae019b68dee580feb34c4947518f33f	2023-07-06	18,854	3,143	3,143	5	
529c96a5221a3bdd41f6075f074a3c98	2023-07-13	59,045	0	0	10	
45b049fff9cf9494c94e49b8020c5e9	2023-10-16	17,200	0	0	7	
9242c770357b68f5b1c70bc3bc9a76a9	2023-10-12	16,086	0	0	10	
3cae1320f97051eaa5b16b42fa56309	2023-06-22	39,256	0	0	6	
4fb65b0852668250aa9a959b936af3f5	2023-07-12	7,945	0	0	3	
0e4ee756800cb246831a31b3f75fbc38	2023-07-28	23,850	0	0	6	

202 rows, 26 columns

No selection

Actually, sometimes both columns are zeroed out; but most of the times, no active loans is reflected in a "-1" in number of days. we can probably just identify the two cases in one.

```
In [ ]: # Numbers of payments < 0
count_where(
    (pl.col("num_confirmed_payments_3m") < 0)
    | (pl.col("num_confirmed_payments_6m") < 0)
    | (pl.col("num_failed_payments_3m") < 0)
    | (pl.col("num_failed_payments_6m") < 0)
    | (pl.col("num_failed_payments_1y") < 0)
)
```

0

```
In [ ]: # Monetary amounts < 0
count_where(
    (pl.col("new_exposure_7d") < 0)
    | (pl.col("new_exposure_14d") < 0)
    | (pl.col("amount_repaid_14d") < 0)
    | (pl.col("amount_repaid_1m") < 0)
    | (pl.col("amount_repaid_3m") < 0)
    | (pl.col("amount_repaid_6m") < 0)
    | (pl.col("amount_repaid_1y") < 0)
)
```

0

```
In [ ]: # Exposure at 14 days lower than exposure at 7 days
count_where(pl.col("new_exposure_14d") < pl.col("new_exposure_7d"))
```

0

```
In [ ]: # Number of payments lower at an earlier date
count_where(
    (pl.col("num_confirmed_payments_6m") < pl.col("num_confirmed_payments_3m")
    | (pl.col("num_failed_payments_6m") < pl.col("num_failed_payments_3m"))
    | (pl.col("num_failed_payments_1y") < pl.col("num_failed_payments_6m"))
)
```

0

```
In [ ]: # Amount repaid higher at an earlier date
count_where(
    (pl.col("amount_repaid_1m") < pl.col("amount_repaid_14d"))
    | (pl.col("amount_repaid_3m") < pl.col("amount_repaid_1m"))
    | (pl.col("amount_repaid_6m") < pl.col("amount_repaid_3m"))
    | (pl.col("amount_repaid_1y") < pl.col("amount_repaid_6m"))
)
```

0